



Towards neuromorphic brain-computer interfaces: Model and circuit Co-design of the spiking EEGNet

Lichen Feng^a, Hongwei Shan^a, Zhouhua Fan^b, Yueqi Zhang^a, Liying Yang^{c,*}, Zhangming Zhu^a

^a School of Microelectronics, Xidian University, Xi'an, China

^b Beijing Institute of Computer Technology and Application, Beijing, China

^c School of Computer Science and Technology, Xidian University, Xi'an, China

ARTICLE INFO

Keywords:

Brain-computer interfaces
Neuromorphic
Activation quantization
Energy efficiency

ABSTRACT

The compact end-to-end deep neural network model, EEGNet, scalable to various brain-computer interface (BCI) paradigms, has been realized in hardware but with low efficiency. Neuromorphic circuit with high efficiency has not yet been applied to BCI. In this paper, the activation-quantization-based (AQB) method that converts EEGNet to spiking neural network is proposed and optimized on the standard motion imagery dataset BCIC-IV-2a with an accuracy of 63.35%. The proposed circuit of the AQB EEGNet is implemented in FPGA, and shows an energy efficiency of 318μJ/inference, which is comparable to that of a low power MCU at 337μJ/inference. Estimation of the energy efficiency in neuromorphic hardware shows the high efficiency of 34μJ/inference, which is a huge improvement compared to previous works, and benefits the BCI for long-term use.

1. Introduction

Brain-computer interfaces (BCIs) enable communication between brain and machine directly [1], which have been used in event-related potential classification [2], intention [3] or emotion recognition [4], and seizure detection [5], etc. With the development of low-cost electrodes [6], wearable BCIs based on scalp electroencephalogram (EEG) signals are attracting much attention.

The great success of deep learning, which learns high-level features end-to-end through raw signal, has renewed interest in BCIs. A variety of artificial neural networks (ANN) models have been proposed for different BCI applications [7–9]. Based on depthwise and separable convolutions, Lawhern et al. [10] proposed a compact and scalable architecture, EEGNet, to classify EEG signals for different paradigms, whose variants (S-EEGNet [11], TSGL-EEGNet [12]) have been developed to improve performance. However, limited by the computational scheme of traditional convolutional neural network (CNN), many multiplying and accumulation (MAC) operations are required, leading to a heavy workload for implementing EEGNet in hardware.

Inspired by the spiking information transmission mode between neurons in the highly-efficient human brain [13], the bio-plausible spiking neural networks (SNNs), which reduce computational complexity by only transmitting binary spike events, has become a

possible substitute for CNNs [14]. To get the spiking version of EEGNet, there are two possible methods in common. On the one hand, the SNN counterpart of EEGNet can be constructed and trained directly through bio-plausible learning methods, such as spike-timing-dependent plasticity [15]. The accuracy obtained through this method is generally low. On the other hand, the well-trained EEGNet with back-propagation can be converted to SNNs directly, to maintain the accuracy.

There have been many researches regarding the efficient conversion of ANN to SNN [16–21]. The efficient Rectified Linear Unit (ReLU)-based conversion method with weight-normalization has been proposed for years [18]. Recently, the temporal pattern coding method [19] is presented to better express the activation. Maass et al. [20] presented an iterative spiking neuron to approximate the activation function in CNN. Liu et al. [21] developed the Spikeconverter which uses mechanism with integration and firing steps, as well as soft-reset method to achieve a better accuracy with less spikes. However, the abovementioned works focus on the task of image classification, where inputs are integers of gray scale within a small range. The multichannel scalp EEG signal with negative values have much more fractional bits to represent details, which makes these methods off-the-shelf [18–21] cannot be applied directly.

In this case, the customized conversion method, activation-based quantization (AQB) method, is proposed for the first time in this

* Corresponding author.

E-mail address: yangliying1208@163.com (L. Yang).

<https://doi.org/10.1016/j.mejo.2023.105808>

Received 1 January 2023; Received in revised form 21 April 2023; Accepted 23 April 2023

Available online 26 April 2023

1879-2391/© 2023 Elsevier Ltd. All rights reserved.

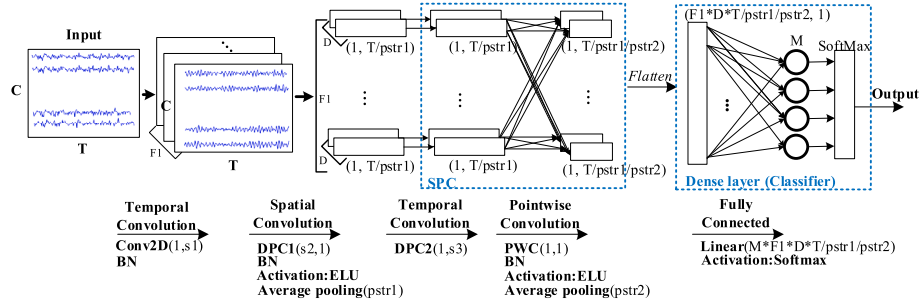


Fig. 1. The architecture of EEGNet.

Table 1
EEGNet parameters.

Parameters	Meaning	In which Layer	Typical value
C	No. Of electrode channels	Input Layer	Selected 16 for SMR
T	No. Of sample points of epochs	Input Layer	256 for SMR
F1	No. Of temporal filters	Conv2D	8
D	No. Of spatial filters	DPC1	2
s1	Kernel length of the temporal filters	Conv2D	32
s2	Kernel length of the temporal filters	DPC2	16
pstr1	Stride of AvePool1	Average Pool 1	4
pstr2	Stride of AvePool2	Average Pool 2	8
Z	No. Of pointwise conv kernels	PWC	16
M	No. Of class	Output Layer	4

paper, to obtain the efficient spiking EEGNet. In specific, by setting the quantization thresholds for neurons, the traditional real-value activations are quantized to spike trains for information transmission like what happens between real neurons. The original EEGNet model is optimized by merging batch normalization into convolution and changing activation function, before spiking conversion. To implement the proposed method efficiently, the model and hardware co-design method [22] is adopted. The simplified spiking convolution block without multipliers is designed according to the AQB spiking EEGNet. The spike-centric convolution fully utilizing the sparsity of the spikes is realized in the spiking convolution blocks. The corresponding address generator is also simplified.

The remaining of this paper is organized as follows. Section 2 describes EEGNet and its parameters in detail. Section 3 proposes the conversion method for EEGNet which is designed hardware-efficiently. The experimental results and discussions are shown in section 4, followed by conclusions in section 5.

2. The compact EEGNet model and its optimization

Firstly, as shown in Fig. 1, the input multichannel EEG epoch with size (1, T) is bandpass-filtered by F1 temporal convolution kernels with size (1, s1) into F1 frequency bands (denoted as Conv2D in Fig. 1). Batch normalization (BN) is used to reduce internal covariate shift. The frequency-specific spatial features are then extracted by the subsequent depthwise convolutional layer (DPC1 in Fig. 1). The parameter D controls the number of spatial filters for each frequency band. BN layer and exponential linear unit (ELU) activation are appended after DPC1. After down-sampling using average pooling of size (1, pstr1), a separable convolutional (SPC in Fig. 1) layer consisting of a depthwise convolution (DPC2 in Fig. 1) and a pointwise convolution (PWC in Fig. 1) is used to complete the temporal summary for each feature map (FM), and optimally merging of the FMs across channel. BN layer and ELU activation

are also appended. A dense layer with the SoftMax activation is designed as the multi-class classifier to generate the output after the down-sampling from average pooling of size (1, pstr2).

More details can be found in Ref. [10]. Table 1 summarizes the parameters of EEGNet for the SMR dataset [23]. The SMR dataset is the standard motion imagery dataset, BCIC-IV-2a. The 288*9 EEG epochs in this dataset are recorded from 9 subjects imaging left, right, foot, and tongue movements. The EEG epoch is resampled to 128 Hz and extracted from [0.5, 2.5]s post cue as a trial. 16-channels are selected using embedded channel selection method [24] to reduce complexity. Therefore, C = 16, T = 256, M = 4 for this dataset.

2.1. Merging BN into convolutional layer

The BN layer in ANNs can reduce internal covariate shift and speed up the training phase. The original computation of this layer is shown in (1).

$$\begin{cases} y_{conv} = W * x + b \\ y_{BN} = \gamma(y_{conv} - \mu) / \sigma + \beta \end{cases} \quad (1)$$

where x and y_{conv} are the input and output of the convolutional layer, respectively. μ and σ are the mean and variance of the mini-batch, respectively. The two learned parameters β and γ are all obtained during training [25].

The repeated computations of BN for each sample is expensive, and is unfriendly to SNN where no BN exists generally. In this case, the BN layer in EEGNet is merged into preceding convolutional layer before the spiking conversion as shown in (2), which is lossless.

$$\begin{cases} W'_l = \frac{\gamma_l}{\sigma_l} * W_l \\ b'_l = \frac{\gamma_l}{\sigma_l} * (b_l - \mu_l) + \beta_l \\ y_{BN} = W'_l * x + b'_l \end{cases} \quad (2)$$

2.2. Changing the ELU activation to ReLU

The ELU activation is used in the original EEGNet, which involves computing exponential function when input activation is negative to obtain high accuracy. In the available conversion algorithms [18–21], the ReLU activation is adopted, where the output of the ReLU is proportional to the number of spikes produced by an IF neuron without refractory period within a given time window. Consequently, in order to successfully convert EEGNet to spiking neural network, and reduce complexity, the activation function of ELU is modified to be ReLU. The simulated accuracies of EEGNet with ELU and ReLU on the dataset of SMR are 68.98% and 66.12%, respectively. An accuracy-drop of 2.86% happens on the SMR dataset, which is moderate. More efforts may be done to improve the accuracy of EEGNet with ReLU activation. We will focus on converting the ReLU-based EEGNet in the following parts.

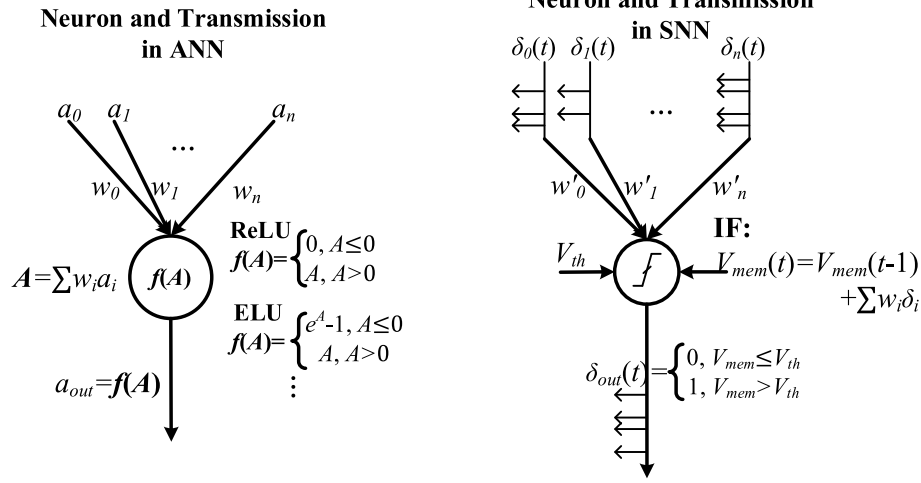


Fig. 2. Comparison between ANN and SNN in terms of model.

2.3. Changing SoftMax function to max

SoftMax layer is applied to predict the probability of every category to obtain the classification result as shown in (3).

$$P(a_i) = \frac{e^{a_i}}{\sum_{i=1}^M e^{a_i}} \quad (3)$$

where a_i is the activation of the i th output neuron, M is the number of output neurons (in the two datasets, $M = 4$, $i = 1, 2, 3, 4$).

The SoftMax function calculates complex exponential function, which is designed to enable the error back-propagation learning when the target function is cross-entropy. In inference phase, however, the SoftMax function would not change the relative order of the M input values. The predicted category of the input sample should be the label of the output neuron with the maximum activation when propagation finishes. On the other hand, the SNN model [26] adopts the label of the neuron with maximum activation or spike number as the predicted result.

Hence, a simple Max function is used to find the neuron with the maximum activation in EEGNet, which has no influence on accuracy but benefits the conversion.

3. The spiking conversion of the optimized EEGNet

3.1. Differences between ANN and SNN

Converting ANN into SNN has been proven to be a promising approach [18–21], since it can obtain a high performance SNN model directly based on the accurate ANN trained by mature learning methods. The main differences between ANN and SNN are the neuron models and the information transmission protocols.

As shown in Fig. 2, the neuron in traditional ANN receives continuous real-values from pre-synapse neurons, and calculates their weighted sum and the corresponding activation function such as ELU, ReLU, etc. In this case, the output of the neuron is also a continuous real-value.

The neuron in SNN receives spike trains from pre-synapse neurons, and update the membrane potential ($V_{mem}(t)$) shown in Fig. 2 according to the model definition. Membrane potential (MP) is another expression of activation. In Fig. 2, a simple Integrate and Fire (IF) spiking neuron model and its update equation are shown. The updated $V_{mem}(t)$ is compared with a threshold (V_{th}) to determine whether to fire a spike. If a spike fires, $V_{mem}(t)$ needs to be reset either to zero or to a certain level (e. g., soft-reset in Ref. [21]). In this case, the output of the neuron is a spike

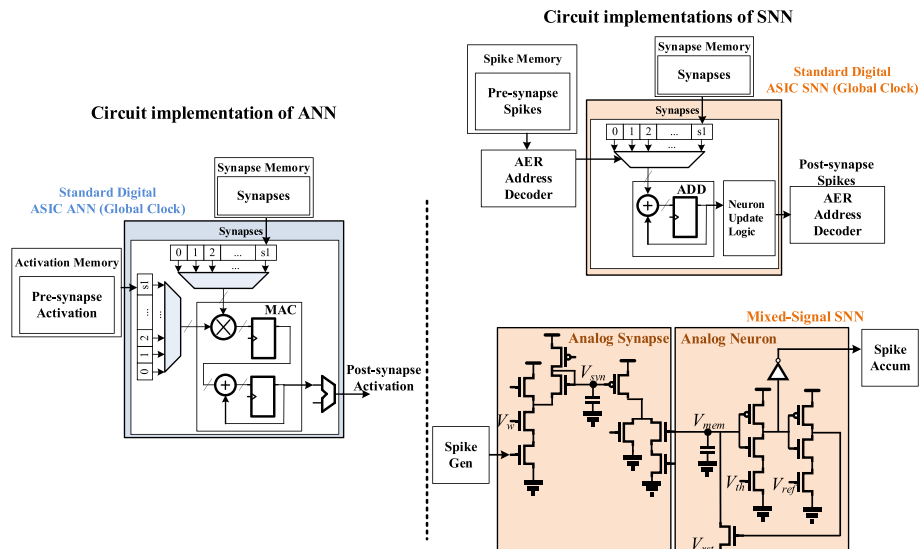


Fig. 3. Comparison between ANN and SNN in terms of circuit implementation.

train.

According to the above discussion, a real-value is propagated to the post-synapse neurons in ANN. A MAC and an activation operation are executed. But in SNN, a spike train is propagated, and additions and comparison are executed. Conversion of ANN to SNN involves converting the activations in ANN to spike trains, migrating the weights, and determining the threshold.

In terms of circuit implementation, as shown in Fig. 3, traditional ANN accelerators [27,28] with multidimensional convolution are mostly implemented through standard digital application specific integrated circuit (ASIC) procedure, in which pre-synaptic real-value activations and synapse weights are stored in memories, and post-synaptic activations are calculated by logic cells operating.

MAC and activation (ReLU for example). Global clock is required to drive the sequential circuits involved. It is hard to tradeoff between resource consumption and performance since the number of MAC operations per convolutional layer is fixed, which can be calculated according to (4) [29].

$$\text{No. MACs} = h_{\text{out}} \times w_{\text{out}} \times n_{\text{out}} \times h_k \times w_k \times n_{\text{in}} / g \quad (4)$$

where h_{out} , w_{out} , and n_{out} correspond to the width, height, and the number of output FM, n_{in} is the number of input FM, h_k and w_k correspond to the shape of synapse kernel. g being the number of groups (commonly $g = 1$, for depthwise separable convolution, $g = n_{\text{in}}$).

The implementation of EEGNet requires millions of MACs (13MMACs for 4.5 s EEG epoch in Q-EEGNet [29]). Full-parallel design would consume huge area and power [30]. Even the quantized EEGNet for optimized implementation in a low-power MCU [29] consume the power of 11.73 mW (90pJ/MAC). For the long-term use of EEGNet near electrodes, low-power consumption designs using CMOS process are being developed. SNN is one of the promising methods.

For implementing SNN, as shown in Fig. 3, there are standard digital ASIC works [31–33] and mixed-signal implementations [34–36]. Compared with digital ANN, digital SNN circuit reduces a multiplier in logic cell since the spike train is propagated and the synapse weights are only added. Address-event representation (AER) decoders are usually required for denoting the location of the spikes. Global clock is still required. As for mixed-signal SNN, the spiking neuron and synapse can be realized by analog circuits. A simple example of the analog neuron and synapse [37] is shown in Fig. 3, where the MP and the voltage level of synapse is hold by a capacitance. The comparisons with threshold and resets of the MP are achieved using transistors. The analog circuits could be event driven which save the power consumption of global clock, but the static current leakage is a problem. However, the energy per synapse operation is still tens of pJ in both digital SNN [32] or mixed-signal SNN [34].

The main advantage of SNN is the computational complexity can be scaled down by making the spike train sparse. The density of the input spike trains determines the number of additions, and determines the density of output spike trains along with the threshold V_{th} . Sparser and shorter spike train would consume fewer hardware resources but may lead to low accuracy. A tradeoff between the length of spike train (number of timesteps) and performance is required before implementation.

In this case, an algorithm converting the real-based propagation in EEGNet to spike-based is proposed accordingly. The energy consumption of the converted SNN algorithm implemented in one of the neuromorphic hardware [31–36] can then be estimated using the energy per synapse operation, which includes the energy consumed when a neuron fires a spike (26 pJ [33]-50 pJ [34]), and the energy when a spike transfer through synapses (5.3 pJ [33]-147 pJ [34]).

3.2. Activation-quantization-based spiking conversion of EEGNet

According to the above discussion, the activation-quantization-based

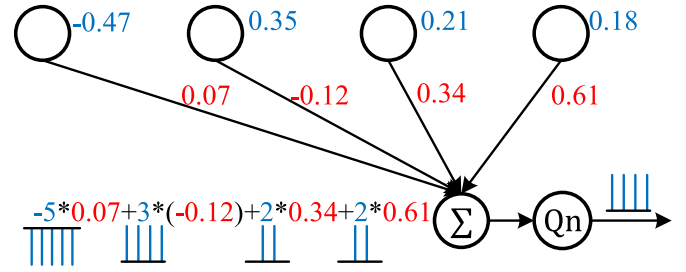


Fig. 4. The AQB computation and propagation mode.

Table 2

The Activation-Quantization-Based spike conversion of EEGNet.

Algorithm: The AQB Spike Conversion of EEGNet
Input: multichannel EEG epoch $X(C, T)$ and weight of network $w(I)$
Output: generated spike trains S of output M neurons
Params: $V_s(I)$ is the quantization threshold in layer l
For layers $l = 1$ **to** L **do**
 for $i = 1$ **to** No. Neurons in layer $l-1$
 $t(l-1, i) = 1$;
 if $FM(l-1, i) > 0$ // Stage 1: fire spikes
 while $(FM(l-1, i, t) - Vs(l-1)) > 0$ **do** // fire positive spikes
 $S^+(l-1, i, t) = 1$;
 $FM(l-1, i) = FM(l-1, i) - Vs(l-1)$;
 $t(l-1, i) = t(l-1, i) + 1$;
 Else
 while $(FM(l-1, i, t) + Vs(l-1)) < 0$ **do** // fire negative spikes
 $S^-(l-1, i, t) = 1$;
 $FM(l-1, i) = FM(l-1, i) + Vs(l-1)$;
 $t(l-1, i) = t(l-1, i) + 1$;
 Endif
 Timestep $Ts(l-1) = \max[t(l-1, i)]$
 End
 for timesteps $t = 1$ **to** Ts **do** // Stage 2: accumulate input with weights
 for $i = 1$ **to** No. Neurons connected to Neuron j in layer l
 if $S^{+/-}(l-1, i, t) = 1$
 $FM(l, j, t) = FM(l, j, t-1) \pm w_{ij}(I)$
 else
 $FM(l, j, t) = FM(l, j, t-1)$
 endif
 endfor
 End
End

(AQB) spike conversion algorithm for EEGNet is proposed, which uses the integrate and fire (IF) neuron model and propagates spike train between neurons.

The IF neuron in a convolutional layer performs two stages. In the first stage, the pre-synapse MP is quantized to fire spikes to the post-synapse neuron. In the second stage, spikes are accumulated to update the MP of the post-synapse neuron, according to the synapse weights. The number of spikes is determined according to

$$N_s(i) = \text{round}\left(\frac{A(i)}{V_s}\right) \quad (5)$$

where $A(i)$ represents the activation of neuron i , V_s represents the quantization threshold, and $N_s(i)$ represents the number of spikes generated by neuron i (its polarity determines the spikes are positive or negative).

According to this equation, the output activation of a neuron is converted to an integer N_s , and then a spike train with N_s spikes in specific polarity is generated for transmission, which simplifies the neuron update logic at the same time.

An example is shown in Fig. 4, where “ Σ ” corresponds to the accumulation stage, and “Qn” the spike firing stage. Assume $V_s = 0.1$, the input activations ($a_{0-3} = -0.47, 0.35, 0.21$, and 0.18) of pre-synapse neurons are quantized to integers ($Int_{0-3} = -5, 3, 2$, and 2) according to (5). The integers can then be encoded into spike trains with

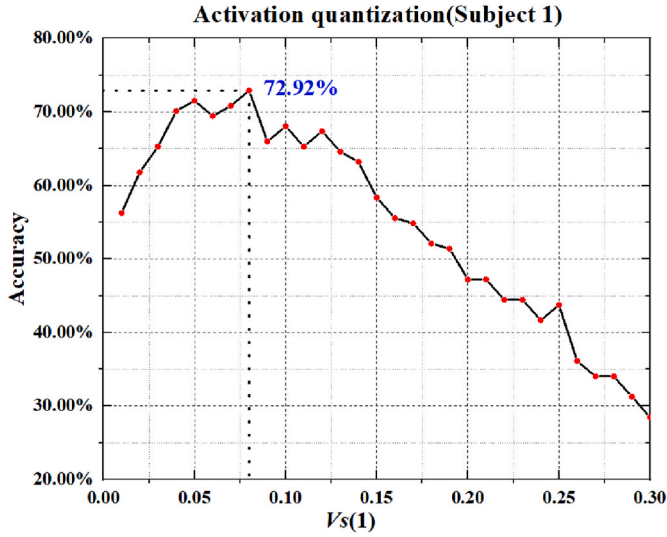


Fig. 5. Accuracy versus $V_s(1)$ in AQB spiking EEGNet.

corresponding number of spikes. The computation of output activation is converted to adding the corresponding weight (real-value) by Int_i times, thus achieving the goal of spike-based transmission.

The pseudocode to realize the AQB conversion of EEGNet is shown in Table 2. The spike conversion of a learned EEGNet is executed layer by layer. By inputting the EEG epoch with size (C, T) , the output of the algorithm is the spike trains generated by the M output neurons. The neurons on pre-synapse FMs are converted into spikes according to (5) by iteratively subtracting the quantization threshold V_s . FM(0) is the input EEG epoch $X(C, T)$. The required timesteps by each neuron is recorded by $t(l, i)$, and the timestep of layer l is the maximum of $t(l, i)$. Positive and negative spike trains are generated separately, and the MP of post-synapse neuron is also updated separately. The FMs are encoded into spike trains with number of spikes proportional to the activation level, which is a kind of rate coding. Note that the proposed AQB spiking EEGNet is not a standard SNN model, since the spikes are generated by quantizing the MP instead of integrate-and-fire neuron model. This method requires an additional spike conversion circuit compared to the digital implementation shown in Fig. 3.

3.3. Determining quantization thresholds and number of timesteps

By designing the operation of two stages, the stochastic error induced by random spikes with poison distribution [17] could be avoided. The main error of the proposed algorithm is the quantization error induced by the threshold V_s , since the fraction bits are abandoned by firing integer number of spikes. Larger V_s leads to lower accuracy which is not intended. Smaller V_s would get better approximation but more spikes will be generated, and if V_s is smaller than a certain level, redundant

information will be transmitted compared with original activation. In this case, V_s in each layer should be determined carefully beforehand since the range of activations in each layer varies.

In the AQB conversion, the spikes are fired by neurons, so the convolution computation and activation function are merged (BN layer has been merged into preceding convolution layer according to (2)). The computation in average pooling layer of original EEGNet is to compute the mean value within the pooling window, as shown in (6)

$$y = \frac{\sum_{i=1}^{N_p} x_i}{N_p} \quad (6)$$

where x and y represent the input and output, respectively. N_p represents the size of pooling window.

By observing (5) and (6), one can conclude that dividing by N_p is an operation that can be.

Merged to quantization process. Therefore, there are five layers in the optimized EEGNet requiring conversion, including Input, conv2D, DPC1, DPC2, PWC layers. Here, we take subject 1 in the SMR dataset as an example to introduce the V_s selection process. The initial values of V_s 's are set to 1 in all layers. At first, in the Input layer, the quantization threshold $V_s(1)$ is swept from 0.01 to 0.30 with the step of 0.01, the other layers are not quantized. The simulation result of inference accuracy versus $V_s(1)$ is shown in Fig. 5 which is in accordance with the above analysis, and $V_s(1)$ can then be determined as 0.08. With the selected $V_s(1)$, the inference accuracy versus $V_s(2)$ can be obtained to determine $V_s(2)$. $V_s(3) \sim V_s(5)$ can then be selected layer by layer.

The corresponding quantization thresholds in each layer for the nine subjects are summarized in Table 3. There are small variations across subject in each layer.

Another important parameter is the maximum number of timesteps. The maximum number of timesteps balances the number of spikes and accuracy. Similar as determining V_s 's, we find the optimal maximum number of timesteps for each subject after fixing the V_s 's. The results are shown in Table 3 along with the thresholds.

4. The spiking convolution circuit for the spiking EEGNet

With the AQB spike conversion method for EEGNet, we propose the circuit design for the efficient implementation of the spiking EEGNet. According to the analysis of EEGNet shown in Fig. 1, four kinds of operations are executed, including temporal convolutions in Conv2D and DPC2, spatial convolution in DPC1, and pointwise convolution in PWC. However, the operations in pointwise convolution are the same as that in spatial convolution in essence, except that multiple convolution channels are combined in pointwise convolution, instead of multiple electrode channels in spatial convolution. Therefore, for implementing the spiking EEGNet, with BN and average pooling layers merged and ELU replaced by ReLU, only two kinds of spiking convolution circuits (temporal and spatial convolution circuits) are designed.

Table 3

The selected quantization thresholds for the nine subjects in SMR dataset.

Subjs.	$V_s(1)$ Conv2D	$V_s(2)$ DPC1	$V_s(3)$ DPC2	$V_s(4)$ PWC	$V_s(5)$ FCL	Acc. EEGNet	Acc. Spike EEGNet	Max. Timesteps
Subj1	0.08	1	0.4	1.2	10	77.78%	72.92%	38
Subj2	0.18	0.9	0.3	1.1	8	43.75%	43.75%	14
Subj3	0.15	1.1	0.4	1.3	12	87.50%	76.39%	25
Subj4	0.14	1.3	0.4	1.5	10	61.81%	61.81%	32
Subj5	0.12	1	0.4	1.2	10	43.75%	43.75%	32
Subj6	0.13	1	0.5	1.3	9	45.83%	44.44%	35
Subj7	0.13	0.9	0.8	1.2	13	82.64%	76.39%	39
Subj8	0.12	1.1	0.5	1.2	5	74.31%	73.61%	18
Subj9	0.15	0.9	0.4	1.2	11	83.33%	77.08%	22
Ave.	—	—	—	—	—	66.74%	63.35%	28

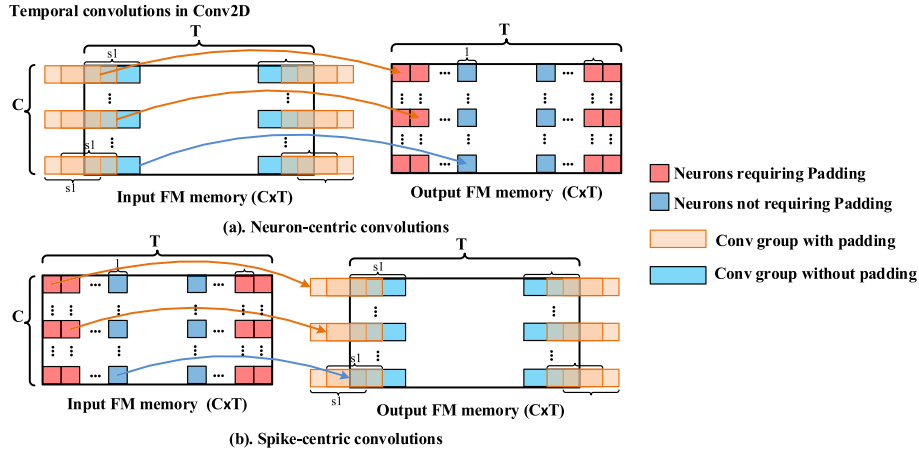


Fig. 6. Comparison between neuron-centric (a) and spike-centric convolution (b).

Spiking Convolution Circuit

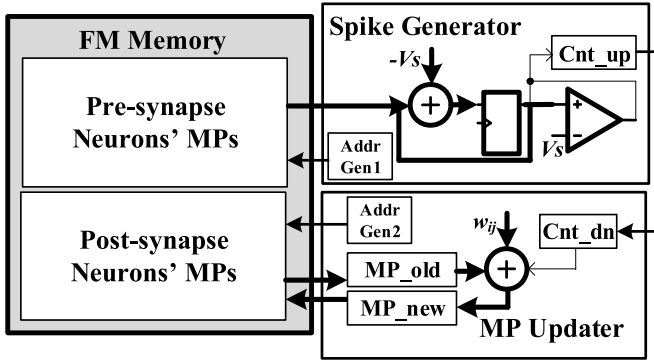


Fig. 7. Design of the spike-centric spiking convolution circuit.

AQB spiking conversion method requires a spike generator converting the MP into spikes in the first stage, and an MP updater updating the potential of post-synapse neurons in the second stage. The adopted spike-centric convolution method and the design of the spiking convolution circuit is described in detail as follows.

4.1. Spiking convolution circuit

Stage 2 in Table 2, accumulating input spikes with weights, is a standard neuron-centric convolution operation. Taking Conv2D as an example, as shown in Fig. 6(a), spike trains fired by $s1$ consecutive pre-synapse neurons in each electrode channel (C electrode channels in total) determine the MP of one post-synapse neuron. Padding operations are required for $s1$ post-synapse neurons. Directly implementing such a convolution requires caching $s1$ spike trains and a complex address generator. In AQB spiking EEGNet, the lengths of the $s1$ cached spike trains are different, which makes the schedule complex.

Hence, the spike-centric convolution as shown in Fig. 6(b) is adopted, in which the spikes fired by one pre-synapse neuron are cached, and the post-synapse neurons influenced by this spike train are all updated

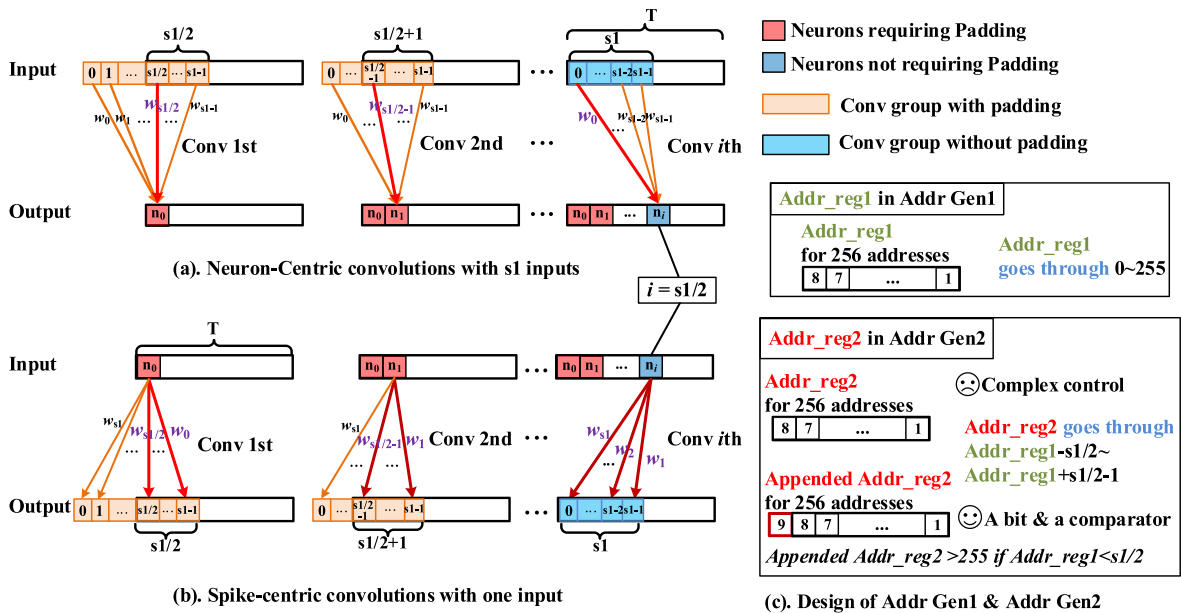


Fig. 8. Connections between input and output neurons in neuron-centric (a) and spike-centric convolutions (b), and the design of Addr Gen1 and Addr Gen2 (in Conv2D).

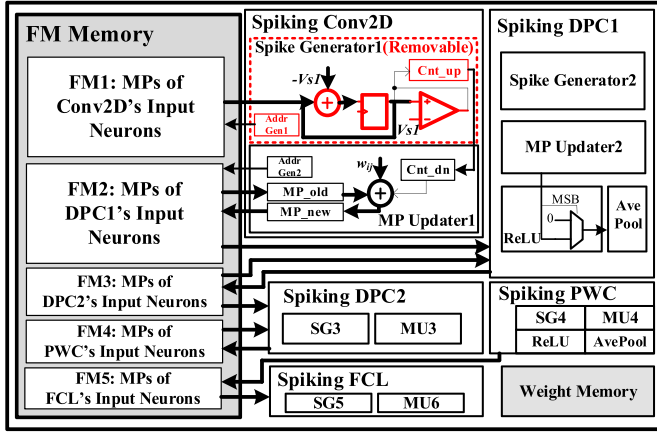


Fig. 9. System architecture design of spiking EEGNet.

before processing another spike train. The control logic and the address generator are then simplified compared to neuron-centric convolution in this case, which also brings about another advantage of adapting ability to the sparse spike train (discussed later). Padding operations are still required.

Based on spike-centric convolution, the spiking convolution circuit, composed of an FM memory, a Spike Generator, and an MP Updater, is proposed as shown in Fig. 7. The FM memory stores the MPs of pre- and post-synapse neurons. The Spike Generator and MP Updater quantize the activation of the pre-synapse neurons and update the MP of the post-synapse neuron, respectively. Details of the circuits are shown as follows.

In one spike-conversion period, the Spike Generator reads the MP of a specific pre-synapse neuron according to the address generated by the unit Addr Gen1. The obtained MP is decreased by a threshold value V_s in each clock cycle, and is compared to V_s to determine whether the counter “Cnt_up” is increased by one. Cnt_up stops counting if MP is smaller than V_s , which is in accordance with the stage 1 shown in Table 2, so the number of spikes generated by the AQB method is stored in Cnt_up. Accordingly, Addr Gen1 generates the address of pre-synapse neuron one-by-one from the start to end sequentially without padding, which is much simpler than that in neuron-centric convolution.

In MP Updater, Cnt_dn counts from Cnt_up to zero, so the MPs of the neurons influenced by this pre-synapse neuron is updated with the synapse weights “Cnt_up” times, which corresponds to accumulating input spikes with weights without multiplication. Due to spike-centric convolution, the logics of “Addr Gen1” and “Addr Gen2” are redesigned.

4.2. Redesigned Addr Gens according to spike-centric convolution

Firstly, in traditional neuron-centric convolutions as shown in Fig. 8 (a), the sliding window of size s_1 (kernel size in Conv2D) is applied on the input feature map. The connections between the first input neuron and the $s_1/2 + 1$ output neurons are annotated in red lines, and the synapse weights in purple. Therefore, all the output neurons that are influenced by the first input neuron are determined as shown in Fig. 8 (b). The weights of these connections are arranged in an inverse order as can be observed. The same method applies to all the remained input neurons, except that the padding neurons are different.

In this case, we have proposed the concise design of Addr Gen1 and Addr Gen2 as shown in Fig. 8. In Addr Gen1, an Addr_reg1 with 8-bit (taking Conv2D layer as an example) is used to annotate the reading address from input FM memory. Addr_reg1 goes through 0–255. For each obtained MP, it is converted to the number of spikes (recorded by Cnt_up) in Spike Generator. The number of spikes is used to update the MPs located by Addr_reg2 of Addr Gen2 in MP updater from the output

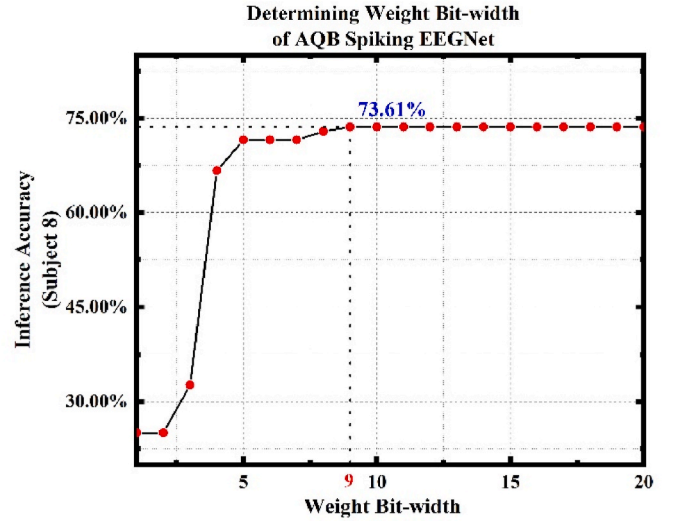


Fig. 10. Inference accuracy versus weight bit-width, taking subject 8 as an example.

FM memory as shown in Fig. 7. Addr_reg2 is generated according to the connections shown in Fig. 8(b), which goes through Addr_reg1- $s_1/2$ to Addr_reg1+ $s_1/2-1$. Since Addr_reg1 goes through 0–255, Addr_reg2 is smaller than 0 if Addr_reg1 < $s_1/2$, and bigger than 255 if Addr_reg1 > $255 - s_1/2 + 1$. In this case, complex control circuit, which compares Addr_reg1 with the two boundary values, calculates the valid Addr_reg2, is required if Addr_reg2 can only represent addresses from 0 to 255. Therefore, an additional bit is appended to Addr_reg2. When Addr_reg1 < $s_1/2$, Addr_reg2 = Addr_reg1- $s_1/2$ is bigger than 255, so the two comparisons could be unified, which simplifies the circuit structure.

4.3. System consideration

According to Fig. 1 and Table 3, the system architecture of spiking EEGNet is the sequential connections of five spiking convolution blocks, along with the five FM memories as shown in Fig. 9. The simple ReLU and Average pooling circuits are added in Spiking DPC1 and PWC blocks.

The difference between the five spiking convolution blocks lies in how the addresses of pre-synapse neurons and post-synapse neurons are calculated. The spatial convolutions in DPC1 and PWC, and the fully-connected connection calculation in FCL have no paddings required. The temporal convolution in DPC2 is with the same principle for address calculation. Therefore, there are no special designs in these blocks.

On the other hand, according to Fig. 1, the memory size of FM1 and FM2 is much larger than that of FM3–5, since DPC1 accomplishes the compression with the ratio of C:1. From the perspective of system implementation, the size of FM1 can be reduced, and the Spike Generator1 in Spiking Conv2D can be removed as shown in Fig. 9, since we can directly store the converted spike numbers in FM1.

5. Experimental results and discussion

The inference accuracy of the AQB spiking EEGNet shown in Table 3 is the result with float-point weights. To accelerate the spiking EEGNet in hardware with negligible loss of accuracy, while minimizing resource utilization, the fixed-point weight is adopted. The weight bit-widths versus the accuracies for the 9 subjects in SMR dataset are simulated to find the optimal weight bit-width. The result on subject 8 is shown in Fig. 10 as an illustrative example, where 9 bits guarantees no loss of accuracy compared to float-point implementation. The weight bit-width of the hardware system is 13 to meet the maximum requirement by the 9

Table 4

Comparison of resource utilization between three block designs.

Resource	Non-spiking Conv2D	Spiking Conv2D with Spike Generator1	Spiking Conv2D without Spike Generator1
LUT	607	413	352
LUTRAM	1	0	0
Flip Flop	1295	591	561
DSP	4	0	0
BRAM	4.5	4.5	3.5
Power Consumption	38 mW	19 mW	16 mW

subjects, so the Weight Memory stores words with 13 bits.

After fixing the bit-width of weights, the bit-widths of feature maps in FM memory are determined by the maximum number of spikes fired by a neuron. According to Table 3, the maximum timesteps is 39 for subject 7, so another 6 bits are added to represent the MPs when no rounding or truncation used. The FM memory stores words with 19 bits, in another word.

To show the effectiveness of removing Spike Generator, the synthesizable Verilog-HDL description of Spiking Conv2D is coded in the determined fixed-pointed format. The kernel weights are trained offline using Tensorflow 2.0, and uploaded in Weight Memory. The coded block is then implemented in the Xilinx KC705 FPGA evaluation board in the environment of Vivado 2017.4, and the resource utilizations of the Spiking Conv2D blocks with and without Spike Generator1, as well as the non-spiking Conv2D are shown in Table 4. The Spiking Conv2D block with Spike Generator has already been 32%, and 54% smaller in terms of LUT and Flip Flop occupation, and 50% lower in terms of power consumption, compared to the non-spiking neuron-centric Conv2D design using the same fixed-point format representation. By directly storing the converted spike numbers that is with lower bit-width than raw EEG signal in FM1, the Spike Generator1 can be removed as discussed above. The LUT number and power consumption are further reduced by 15%, 16%, respectively. The occupied BRAM is also reduced

from 4.5 to 3.5.

To show the advantages of spike-centric convolutions, the spike number of each layer is counted to show the sparsity of the network after AQB spiking conversion. The quantized spike numbers of neurons on four convolution's FMs of a typical EEG sample is shown in Fig. 11. There are hardly zeros on the input FM of Conv2D as shown in top left, but the remaining three FMs after DPC1 are all with sparsity around 50%.

In order to clearly observe the sparsity of each layer, the notation, number of spikes per neuron (nspn), is defined according to (7).

$$nspn = \frac{\sum_{i=1}^{N_{fmx}} ns_i}{N_{fmx}} \quad (7)$$

where N_{fmx} represents the total number of neurons on input FM of x th layer, ns_i represents the number of spikes on the i th neuron during inference.

The averaged nspns' of the 9 subjects are shown by layer in Fig. 12. In this case, the required number additions by each convolution can be calculated according to (8).

$$\text{No. ADDs} = h_{in} \times w_{in} \times n_{in} \times npsn \times h_k \times w_k \times n_k \quad (8)$$

The total 6.2 M ADDs of the AQB spiking EEGNet can be then be derived. The total 1.1 MACs required by traditional implementation of EEGNet with input EEG epochs of size (16, 256) can also be estimated according to (4). Although compared to traditional EEGNet, the circuit design of AQB spiking EEGNet is 32%, and 54% smaller in terms of LUT and Flip Flop occupation, and 50% lower in terms of power consumption, the $5 \times$ operations lead to $5 \times$ processing delay, which degrade the advantages of spiking EEGNet. After observing the npsn shown in Fig. 12, one can conclude that the Conv2D layer which fires 5.87 spikes per neuron is the cause of the long delay. Hence, the better choice is to implement the Conv2D layer using traditional ANN method. In this case, the 98,816 MACs required by traditional DPC1, DPC2, PWC, and FCL layers, can be replaced by 43,264 ADDs by spiking layers, which is 57%

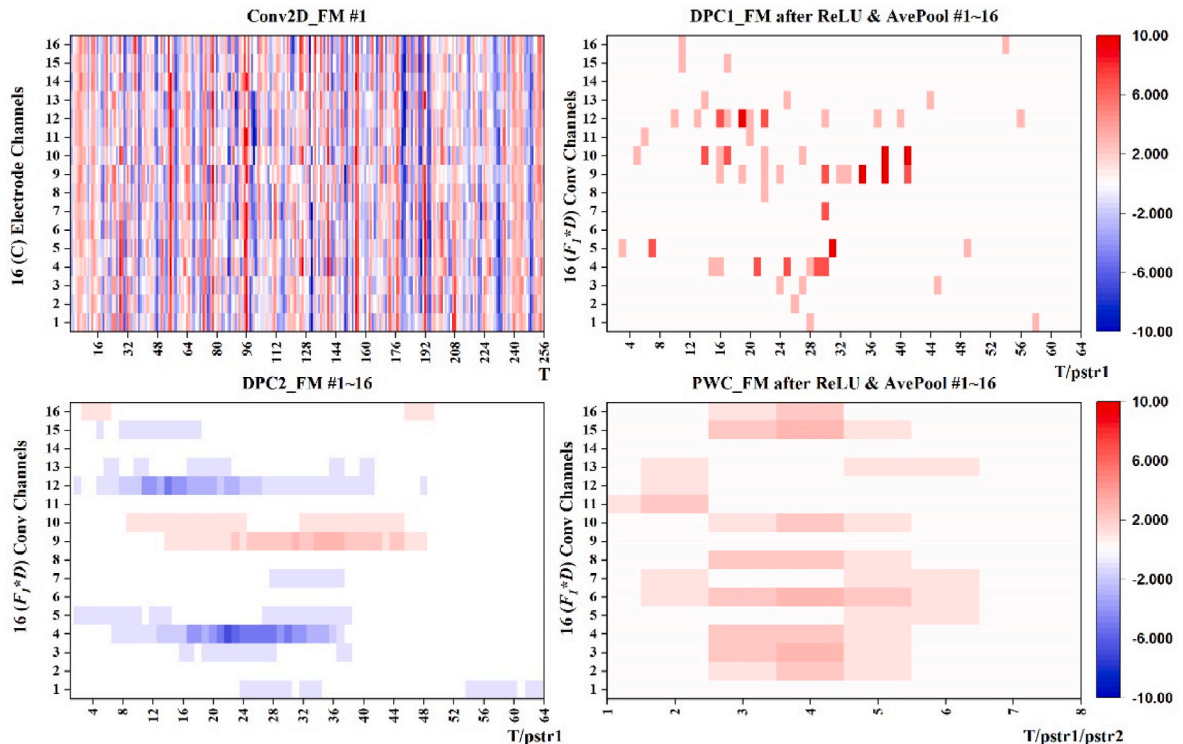


Fig. 11. Quantized spike numbers of neurons on four FMs of a typical EEG sample.

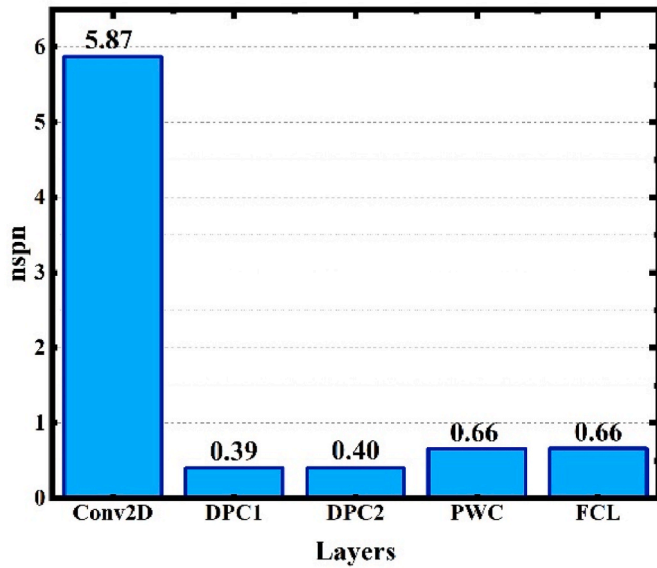


Fig. 12. Quantized spike numbers of neurons on the input FMs for each layer (counted from a typical EEG sample).

improvement in terms of processing speed at the same clock frequency.

As abovementioned, the energy consumption of the spiking EEGNet implemented in the neuromorphic hardware [31–37] can be estimated according to the energy consumed when neuron fires a spike (26 pJ [33]–50 pJ [34]), and the energy when spikes transfer through synapses (5.3 pJ [33]–147 pJ [34]). The number of spikes in total and the number of spike transmissions in total can be calculated separately, and the power consumption per inference of spiking EEGNet in neuromorphic hardware can be estimated as 34–916 μ J/inference accordingly. In terms of processing delay, neuromorphic implementations realizing parallel neuron connections [34–36] are inherently fast, since multiple spikes can be received by neurons at the same time.

The comparisons between previous works using traditional EEGNet, the AQB EEGNet in FPGA, and the AQB EEGNet in neuromorphic hardware are shown in Table 5. In terms of resource utilization and latency, the AQB EEGNet in FPGA needs much less LUTs than the fully parallel implementation in Ref. [30], but is with much lower speed. The energy efficiency of 318 μ J/inference is 46% lower than that in Ref. [30] and is comparable to that in Ref. [29]. The energy efficiency of AQB EEGNet in neuromorphic hardware is estimated as illustrated above. The latency is estimated according to the clock frequency 13 MHz in

Ref. [32]. The maximum timestep, 39, and the number of convolution layers, 5, is as shown in Table 3. For the comparisons of energy efficiency between different processes and clock frequency, the normalized energy to 40 nm CMOS process is calculated as $E_{norm} = E \times (40/\text{process}) \times 50/F_{clk}$, according to the newly published SNN integrated circuit work [38]. The normalized energy efficiency of 91 μ J/inference is 73% lower than the state-of-the-art result 337 μ J/inference shown in Ref. [29].

6. Conclusion

In this paper, the activation-quantization-based (AQB) ANN-to-SNN conversion method that converts EEGNet to SNN is proposed for the first time. The original EEGNet model is optimized by merging batch normalization into convolution and changing activation function before spiking conversion, which gets the accuracy of 63.35%, respectively, on the three BCI paradigms. The simplified spike-centric spiking convolution circuit with customized address generator, and without multipliers is designed to fully utilize the sparsity of the spikes. The proposed circuit is implemented in FPGA with the energy efficiency of 318 μ J/inference, which is comparable to the 337 μ J/inference in a low power MCU. Estimation of the energy efficiency in neuromorphic hardware shows the high efficiency of 34 μ J/inference and the normalized energy efficiency of 91 μ J/inference, which is 73% compared to previous works.

Author statement

I have made substantial contributions to the conception or design of the work: or the acquisition, analysis, or interpretation of data for the work. And I have drafted the work or revised it critically for important intellectual content. And I agree to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

All people who have made substantial contributions to the work reported in the manuscript, including those who provided editing and writing assistance but who are not authors, are named in the acknowledgements.

Thank you for giving us an opportunity to rephrase our paper. We have considered the circuit design, and made necessary changes according to the reviewers' comments. In addition, the reviewers' question had been answered in detail, and we had highlighted the revised parts in our manuscript. We hope that the revised paper will be accepted.

Declaration of competing interest

The authors declare that they have no known competing financial

Table 5

Comparison between previous works, AQB EEGNet in FPGA, and AQB EEGNet in neuromorphic hardware.

Performance	[29]	[30]	AQB EEGNet in FPGA	AQB EEGNet in Neuromorphic Hardware*
Platform	MCU 40 nm LP CMOS	Xilinx VC707 FPGA	Xilinx KC705 FPGA	Neuromorphic Chip 28 nm CMOS
Accuracy/Dataset	70.9%/SMR	88.75%/ERP 76.33%/ERN	63.35%/SMR 82.7%/ERP 76.4%/ERN	
Frequency	50–350 MHz	100 MHz	100 MHz	13 MHz (76.9ns interval between spikes)
Resource Utilization	10 mm ^{2#}	207,896 LUTs (60 channels) 147,427 FFs 2492 DSPs 180 BRAMs	1820 LUTs (2075 with ANN Conv2D) 2805 FFs (2729 with ANN Conv2D) 0 DSPs 63 BRAMs	23–450 μ m ^{2#}
Latency	28.67–31.98 ms	58.84–115.85 μ s	56.0 ms (11.58 ms with ANN Conv2D)	15 μ s*
Energy/Inference [Normalized to 40 nm]	337 μ J	588 μ J	318 μ J (100 μ J with ANN Conv2D)	34–916 μ J* [91–2466 μ J]

The total area of the Chip.

The estimated results according to [33,34].

interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

This work is supported by the National Natural Science Foundation of China (62104175, 61974109).

References

- [1] J.R. Wolpaw, N. Birbaumer, D.J. McFarland, G. Pfurtscheller, T.M. Vaughan, Brain-computer interfaces for communication and control, *Clin. Neurophysiol.* 113 (6) (2002) 767–791.
- [2] M. Xu, J. Han, Y. Wang, T. Jung, D. Ming, Implementing over 100 command codes for a high-speed hybrid brain-computer interface using concurrent P300 and SSVEP features, *IEEE Trans. Biomed. Eng.* 67 (11) (2020) 3073–3082.
- [3] Y. Zhang, C.S. Nam, G. Zhou, J. Jin, X. Wang, et al., Temporally constrained sparse group spatial patterns for motor imagery BCI, *IEEE Trans. Cybern.* 49 (9) (2019) 3322–3332.
- [4] T. Zhang, W. Zheng, Z. Cui, Y. Zong, Y. Li, Spatial-temporal recurrent neural network for emotion recognition, *IEEE Trans. Cybern.* 49 (3) (2019) 839–847.
- [5] M. Tohidi, J.K. Madsen, F. Moradi, A low-power, low-noise, high-accurate epileptic-seizure detection system for wearable applications, *Microelectron. J.* 92 (2019), 104600–104600.
- [6] P. Sawangjai, S. Hompoonsup, P. Leelaarporn, S. Kongwudhikunakorn, T. Wilaiprasitporn, Consumer grade EEG measuring sensors as research tools: a review, *IEEE Sensor. J.* 20 (8) (2020) 3996–4024.
- [7] R.T. Schirmer, J.T. Springenberg, L.D.J. Fiederer, M. Glasstetter, K. Eggenberger, M. Tangermann, F. Hutter, W. Burgard, T. Ball, Deep learning with convolutional neural networks for EEG decoding and visualization, *Hum. Brain Mapp.* 38 (11) (2017) 5391–5420.
- [8] T. Ma, H. Li, H. Yang, X. Lv, P. Li, T. Liu, D. Yao, P. Xu, The extraction of motion-onset vep BCI features based on deep learning and compressed sensing, *J. Neurosci. Methods* 275 (2017) 80–92.
- [9] Y.R. Tabar, U. Halici, A novel deep learning approach for classification of EEG motor imagery signals, *J. Neural. Eng.* 14 (2017), 016003.
- [10] V.J. Lawhern, A.J. Solon, N.R. Waytowich, S.M. Gordon, C.P. Hung, B.J. Lance, EEGNet: a compact convolutional network for EEG-based brain-computer interfaces, *J. Neural. Eng.* 15 (5) (2018) 056013.1–056013.17.
- [11] W. Huang, Y. Xue, L. Hu, H. Liuli, S-Eegnet, Electroencephalogram signal classification based on a separable convolution neural network with bilinear interpolation, *IEEE Access* 8 (2020) 131636–131646.
- [12] X. Deng, B. Zhang, N. Yu, et al., Advanced TSGL-EEGNet for motor imagery EEG-based brain-computer interfaces, *IEEE Access* 9 (2021) 25118–25130.
- [13] W. Maass, Computing with spikes, Special issue on Found. Inform. Process. *TELEMATIK* 8 (1) (2002) 32–36.
- [14] P.A. Merolla, J.V. Arthur, R. Alvarez-Icaza, et al., A million spiking-neuron integrated circuit with a scalable communication network and interface, *Science* 345 (2014) 668–673.
- [15] H. Markram, J. Lübke, M. Frotscher, B. Sakmann, Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs, *Science* 275 (5297) (1997) 213–215.
- [16] Y. Cao, Y. Chen, D. Khosla, Spiking deep convolutional neural networks for energy-efficient object recognition, *Int. J. Comput. Vis.* 113 (1) (2015) 54–66.
- [17] B. Rueckauer, I.A. Lungu, Y. Hu, M. Pfeiffer, S.C. Liu, Conversion of continuous-valued deep networks to efficient event-driven networks for image classification, *Front. Neurosci.* 11 (2017) 682.
- [18] P.U. Diehl, D. Neil, J. Binas, et al., Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing, *IEEE Intern. Joint Conf. Neural Netw. (IJCNN)* (2015) 1–8.
- [19] B. Rueckauer, S. Liu, Temporal pattern coding in deep spiking neural networks, *IEEE Intern. Joint Conf. Neural Netw. (IJCNN)*. (2021) 1–8.
- [20] C. Stöckl, W. Maass, Optimized spiking neurons can classify images with high accuracy through temporal coding with two spikes, *Nat. Mach. Intell.* 3 (3) (2021) 230–238.
- [21] F. Liu, W. Zhao, Y. Chen, et al., Spikeconverter: an efficient conversion framework zipping the gap between artificial neural networks and spiking neural networks, *Proc. AAAI Conf. Artif. Intell.* 36 (2) (2022) 1692–1701.
- [22] W. Byun, M. Je, J.-H. Kim, Advances in wearable brain-computer interfaces from an algorithm-hardware co-design perspective, *IEEE Trans. Circuits Syst. II: Express Briefs* 69 (7) (2022) 3071–3077.
- [23] M. Tangermann, K.R. Müller, A. Aertsen, et al., Review of the BCI competition IV, *Front. Neurosci.* 6 (2012) 55.
- [24] M.Z. Baig, N. Aslam, H. Shum, Filtering techniques for channel selection in motor imagery EEG applications: a survey, *Artif. Intell. Rev.* 53 (3) (2020) 1207–1232.
- [25] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift. *International conference on machine learning, pmlr* (2015) 448–456.
- [26] M. Mozafari, M. Ganjtabesh, A. Nowzari-Dalini, et al., Bio-inspired digit recognition using reward-modulated spike-timing-dependent plasticity in deep convolutional networks, *Pattern Recogn.* 94 (2019) 87–95.
- [27] Y. Zhang, Q. Li, H. Zhang, W. Wang, J. Han, X. Zeng, X. Cheng, A 28 nm, 397uW real-time dynamic gesture recognition chip based on RISC-V processor, *Microelectron. J.* 116 (2021), 105219.
- [28] X. Li, H. Huang, T. Chen, et al., A hardware-efficient computing engine for FPGA-based deep convolutional neural network accelerator, *Microelectron. J.* 128 (2022), 105547.
- [29] T. Schneider, et al., Q-EEGNet: an energy-efficient 8-bit quantized parallel EEGNet implementation for edge motor-imagery brain-machine interfaces, *Proc. IEEE Int. Conf. Smart Comput.* (2020) 284–289.
- [30] A. Tsukahara, Y. Anzai, K. Tanaka, Y. Uchikawa, A design of EEGNet-based inference processor for pattern recognition of EEG using FPGA, *IEEJ Trans. Electronics, Inform. Syst.* 104 (2021) 53–64.
- [31] M.F. Tolba, H.A. Elsafty, M. Armanios, et al., Synchronization and FPGA realization of fractional-order Izhikevich neuron model, *Microelectron. J.* 89 (2019) 56–69.
- [32] C. Frenkel, M. Lefebvre, J.-D. Legat, D. Bol, A 0.086-mm² 12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS, *IEEE Trans. Biomed. Circuits Syst.* 13 (1) (2019) 145–158.
- [33] C. Frenkel, G. Indiveri, ReckOn: A 28nm Sub-mm² Task-Agnostic Spiking Recurrent Neural Network Processor Enabling On-Chip Learning over Second-Long Timescales, *IEEE Intern. Solid-State Circuit Conf. (ISSCC)*, 2022.
- [34] N. Qiao, G. Indiveri, Scaling mixed-signal neuromorphic processors to 28 nm FD-SOI technologies, *Proc. IEEE Biomed. Circuits Syst. Conf.* (2016) 552–555.
- [35] F.N. Buhler, et al., A 3.43TOPS/W 48.9pJ/pixel 50.1nJ/classification 512 analog neuron sparse coding neural network with on-chip learning and classification in 40nm CMOS, *Symp. VLSI Circuits* (2017) C30–C31.
- [36] J. Schemmel, et al., Accelerated Analog Neuromorphic Computing, 2020 *arXiv preprint arXiv:2003.11996*.
- [37] A. Amirshahi, M. Hashemi, ECG classification algorithm based on STDP and R-STDP neural networks for real-time monitoring on ultra-low-power personal wearable devices, *IEEE Trans. Biomed. Circuits Syst.* 13 (6) (2019) 1483–1493.
- [38] X. Qi, J. Zhao, Y. Lou, et al., A 5.3pJ/Spike CMOS neural array employing time-modulated axon-sharing and background mismatch calibration techniques, in: *IEEE Trans. Biomed. Circuits Syst., Early Access*, 2023.